

Homework 09 - Java Collections Framework

Topics Covered

- Collection Framework
- `Collection` vs `Collections`
- Generics
- Interface reference
- `List`
- `ArrayList`
- `LinkedList`
- `Vector`
- `Set`
- `HashSet`
- `LinkedHashSet`
- `TreeSet`
- `Queue`
- `PriorityQueue`
- `ArrayDeque`
- `Map`
- `HashMap`
- `LinkedHashMap`
- `TreeMap`
- `equals()` and `hashCode()`
- `Comparable`
- `Comparator`
- `Iterator`
- `ConcurrentModificationException`
- Choosing the right collection

Instructions

1. Write every program in a separate Java file.
2. First write the expected output, then run the program.
3. For every coding question, write why you selected that collection.
4. For Set and Map questions with custom objects, explain `equals()` and `hashCode()`.
5. Do not use only `ArrayList` everywhere. Select collection based on requirement.
6. After every program, write 2-3 lines explaining real-time use.

Part A: Short Answer Questions

1. Why do we need Collections when arrays already exist?
2. What is the difference between `Collection` and `Collections` ?

3. Why do we use generics in Collections?
4. Why can we write `ArrayList<Integer>` but not `ArrayList<int>` ?
5. Why do professional developers write `List<String> names = new ArrayList<>();` ?
6. What is the difference between `List` , `Set` , `Queue` , and `Map` ?
7. Which collection allows duplicates and maintains insertion order?
8. Which collection removes duplicates but does not guarantee order?
9. Which collection removes duplicates and keeps insertion order?
10. Which collection removes duplicates and keeps sorted order?
11. Which collection is best for key-value lookup?
12. Which Map maintains insertion order?
13. Which Map stores keys in sorted order?
14. What is the internal data structure of ArrayList?
15. What is the internal data structure of LinkedList?
16. What happens internally when ArrayList becomes full?
17. Why is ArrayList fast for `get(index)` ?
18. Why is LinkedList slower for index-based access?
19. What is the internal working of HashSet?
20. Why are `equals()` and `hashCode()` important for HashSet?
21. What is a HashMap bucket?
22. What is collision in HashMap?
23. What is load factor in HashMap?
24. What is the default load factor of HashMap?
25. What happens when HashMap crosses threshold?
26. What is Comparable?
27. What is Comparator?
28. What is the difference between Comparable and Comparator?
29. What is Iterator?
30. Why can `ConcurrentModificationException` occur?

Part B: Choose The Correct Collection

For each situation, write:

1. Collection name
2. Reason
3. Small example

Question 1

Store list of students in a batch.
Order matters.
Duplicate names are allowed.

Question 2

Store unique email ids from registration form.
Order does not matter.

Question 3

Remove duplicate uploaded emails but keep original upload order.

Question 4

Store unique roll numbers in sorted order.

Question 5

Store productId and Product object for fast search by id.

Question 6

Store monthly sales where month keys should be sorted.

Question 7

Process support tickets by priority.

Question 8

Implement browser back button behavior.

Question 9

Store API response data in the same order as database query returned.

Question 10

Count frequency of each word in a paragraph.

Part C: Find The Output

Question 1

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>();

        names.add("Rahul");
        names.add("Priya");
        names.add("Rahul");

        System.out.println(names);
        System.out.println(names.get(1));
    }
}
```

Question 2

```
import java.util.HashSet;
import java.util.Set;

public class Main {
    public static void main(String[] args) {
        Set<String> names = new HashSet<>();

        names.add("Rahul");
        names.add("Priya");
        names.add("Rahul");

        System.out.println(names.size());
        System.out.println(names);
    }
}
```

Write:

Is output order guaranteed?

Question 3

```
import java.util.LinkedHashSet;
import java.util.Set;

public class Main {
    public static void main(String[] args) {
        Set<String> names = new LinkedHashSet<>();

        names.add("Rahul");
        names.add("Priya");
        names.add("Amit");
        names.add("Rahul");

        System.out.println(names);
    }
}
```

Question 4

```
import java.util.Set;
import java.util.TreeSet;

public class Main {
    public static void main(String[] args) {
        Set<Integer> numbers = new TreeSet<>();

        numbers.add(50);
        numbers.add(10);
        numbers.add(30);
        numbers.add(10);

        System.out.println(numbers);
    }
}
```

Question 5

```
import java.util.HashMap;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        Map<String, Integer> marks = new HashMap<>();

        marks.put("Rahul", 85);
        marks.put("Priya", 92);
        marks.put("Rahul", 95);

        System.out.println(marks.get("Rahul"));
        System.out.println(marks.size());
    }
}
```

Question 6

```
import java.util.LinkedHashMap;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        Map<Integer, String> map = new LinkedHashMap<>();

        map.put(103, "Amit");
        map.put(101, "Rahul");
        map.put(102, "Priya");

        System.out.println(map);
    }
}
```

Question 7

```
import java.util.Map;
import java.util.TreeMap;

public class Main {
    public static void main(String[] args) {
        Map<Integer, String> map = new TreeMap<>();

        map.put(103, "Amit");
        map.put(101, "Rahul");
        map.put(102, "Priya");

        System.out.println(map);
    }
}
```

Question 8

```
import java.util.PriorityQueue;
import java.util.Queue;

public class Main {
    public static void main(String[] args) {
        Queue<Integer> queue = new PriorityQueue<>();

        queue.offer(50);
        queue.offer(10);
        queue.offer(30);

        System.out.println(queue.poll());
        System.out.println(queue.poll());
        System.out.println(queue.poll());
    }
}
```

Question 9

```
import java.util.ArrayDeque;
import java.util.Deque;

public class Main {
    public static void main(String[] args) {
        Deque<String> stack = new ArrayDeque<>();

        stack.push("Page-1");
        stack.push("Page-2");
        stack.push("Page-3");

        System.out.println(stack.pop());
        System.out.println(stack.peek());
    }
}
```

Question 10

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Integer> numbers = new ArrayList<>();

        numbers.add(30);
        numbers.add(10);
        numbers.add(20);

        Collections.sort(numbers);
        Collections.reverse(numbers);

        System.out.println(numbers);
    }
}
```

Part D: Find And Fix The Problem

Question 1

```
ArrayList<int> numbers = new ArrayList<>();
```

Write:

1. What is the error?
2. Why primitive type is not allowed in generics?
3. Correct the code.

Question 2

```
import java.util.HashSet;
import java.util.Set;

class Student {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
}

public class Main {
    public static void main(String[] args) {
        Set<Student> students = new HashSet<>();

        students.add(new Student(101, "Rahul"));
        students.add(new Student(101, "Rahul"));

        System.out.println(students.size());
    }
}
```

Write:

1. Why can output be 2 ?
2. What is missing in Student class?
3. Correct using `equals()` and `hashCode()` based on `id` .

Question 3

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>();

        names.add("Rahul");
        names.add("Priya");
        names.add("Amit");

        for (String name : names) {
            if (name.equals("Priya")) {
                names.remove(name);
            }
        }
    }
}
```

Write:

1. What exception can occur?
2. Why does it happen?
3. Correct using Iterator.

Question 4

```
import java.util.Set;
import java.util.TreeSet;

class Product {
    int id;
    String name;
}

public class Main {
    public static void main(String[] args) {
        Set<Product> products = new TreeSet<>();
        products.add(new Product());
    }
}
```

Write:

1. What problem can happen?
2. Why does TreeSet need sorting logic?
3. Correct using Comparator.

Question 5

```
Map<Integer, String> map = new HashMap<>();
map.put(101, "Rahul");
map.put(101, "Priya");
```

Write:

1. How many entries are stored?
2. Which value remains?
3. Why?

Part E: Coding Practice

Program 1: Student List

Create a `Student` class with:

- `id`
- `name`
- `marks`

Create an `ArrayList<Student>`.

Add 5 students.

Print all students using enhanced for loop.

Program 2: Search Student By Id

Using the same student list, search student by id.

If found, print student details.

If not found, print:

```
Student not found
```

Write:

```
Is ArrayList the fastest structure for id lookup?  
Which collection would be better?
```

Program 3: Unique Emails

Create a list with duplicate email ids.

Remove duplicates using `HashSet`.

Print unique emails.

Write:

```
Is order guaranteed?
```

Program 4: Unique Emails With Order

Repeat Program 3 using `LinkedHashSet`.

Print emails in original order.

Program 5: Sorted Roll Numbers

Create a `TreeSet<Integer>` for roll numbers.

Add roll numbers in random order.

Print sorted roll numbers.

Program 6: Product Lookup Using HashMap

Create `Product` class with:

- `id`
- `name`
- `price`

Create:

```
Map<Integer, Product> products = new HashMap<>();
```

Store 5 products.

Search product by id.

Program 7: Word Frequency Counter

Given:

```
java spring java sql spring java
```

Use `HashMap<String, Integer>` to count frequency.

Expected output idea:

```
java = 3
spring = 2
sql = 1
```

Program 8: Character Frequency Counter

Given:

```
banana
```

Use `HashMap<Character, Integer>` to count each character.

Program 9: Shopping Cart

Create a shopping cart using:

```
List<Product>
```

Add 4 products.

Calculate total bill.

Then explain:

```
Why List is suitable here?
```

Program 10: Support Ticket Priority

Create class `SupportTicket` with:

- `ticketId`
- `title`
- `priority`

Use `PriorityQueue<SupportTicket>`.

Lower priority number means higher priority.

Process tickets based on priority.

Program 11: Browser History

Use `ArrayDeque<String>` as stack.

Push:

Use `pop()` to go back.

Print current page after going back.

Program 12: Sort Students By Marks

Create `Student` class with:

- `id`
- `name`
- `marks`

Use `Comparator` to sort students by marks ascending.

Then sort by marks descending.

Program 13: Sort Products By Price Then Name

Create `Product` class with:

- `id`
- `name`
- `price`

Sort products:

1. By price ascending
2. If price is same, by name alphabetically

Use `Comparator`.

Program 14: Comparable Practice

Create `Employee` class implementing `Comparable<Employee>`.

Natural sorting should be by employee id.

Create list of employees and sort using:

```
Collections.sort(employees);
```

Program 15: Course Management Mini Data

Use different collections:

1. `ArrayList<String>` for enrolled students
2. `HashSet<String>` for unique emails
3. `HashMap<Integer, String>` for course id and course name
4. `TreeSet<Integer>` for sorted roll numbers
5. `LinkedHashSet<String>` for uploaded emails without duplicates but same order

Print all data.

Part F: Internal Working Questions

Write explanation in your own words.

1. How does ArrayList grow internally?
2. What is the difference between size and capacity?
3. Why is inserting in middle of ArrayList slower?
4. How does LinkedList store data internally?
5. Why is LinkedList slower for `get(index)` ?
6. How does HashSet check duplicates?
7. Why does HashSet need `hashCode()` and `equals()` ?
8. How does HashMap store key-value pairs?
9. What is a bucket in HashMap?
10. What is collision in HashMap?
11. What is load factor?
12. What happens during HashMap resizing?
13. Why is TreeMap sorted?
14. Why does PriorityQueue use heap?
15. Why can HashMap be faster than TreeMap for lookup?

Part G: Difference Questions

Write differences:

1. Array vs ArrayList
2. ArrayList vs LinkedList
3. List vs Set
4. HashSet vs LinkedHashSet
5. HashSet vs TreeSet
6. HashMap vs LinkedHashMap
7. HashMap vs TreeMap
8. Queue vs Stack
9. PriorityQueue vs normal Queue
10. Comparable vs Comparator
11. Iterator vs enhanced for loop
12. Collection vs Collections

Part H: Real-World Thinking Questions

1. In a login system, where can we use `HashMap` ?
2. In a course platform, where can we use `ArrayList` ?
3. In uploaded CSV email validation, why can `LinkedHashSet` be useful?
4. In a support system, why can `PriorityQueue` be useful?
5. In a report sorted by date, why can `TreeMap` be useful?
6. In product search result, why is `ArrayList` usually enough?
7. In role management, why is `HashSet` useful?

8. In API response, why might order matter?
9. Why should we not select collections by habit?
10. Why is understanding internal working important for backend developers?

Part I: Mini Project

Create a small ****Student Course Management System****.

Classes:

```
Student
- id
- name
- email
- marks

Course
- id
- name
- students
```

Use these collections:

```
ArrayList<Student> -> store students in a course
HashSet<String> -> prevent duplicate emails
HashMap<Integer, Student> -> search student by id
TreeSet<Integer> -> sorted marks
PriorityQueue<Student> -> process lowest marks students first for extra class
```

Requirements:

1. Add at least 5 students.
2. Reject duplicate emails.
3. Search student by id.
4. Print students sorted by marks using Comparator.
5. Print unique emails.
6. Process students for extra class based on lowest marks first.
7. Explain why each collection was used.

Submission Checklist

Before submitting, check:

- Did you choose collection based on requirement?
- Did you avoid using ArrayList everywhere?
- Did you explain internal working where asked?
- Did you use wrapper classes instead of primitive types in generics?
- Did you override `equals()` and `hashCode()` for custom object Set/Map questions?
- Did you handle ordering correctly using LinkedHashSet, LinkedHashMap, TreeSet, or TreeMap?
- Did you use Comparator for custom sorting?

- Did you write expected output before running output questions?
- Did you write real-time use after coding questions?

